Université de Rouen Normandie
Master 1 — Génie Informatique et Logiciel

Système de Gestion de Flotte

Architecture Microservices

Rapport de Semaine 1

Définition, Conception et Fondations du Projet

INFORMATIONS DU PROJET	
Étudiants	[MOUHOUS SIDALI — KHADRAOUI Younes]
Groupe	[Groupe 1 GIL]
Encadrants	Lydia & Luc
Date de rendu	12/03/2026
Année	2025 – 2026

1. Objectifs et Livrables

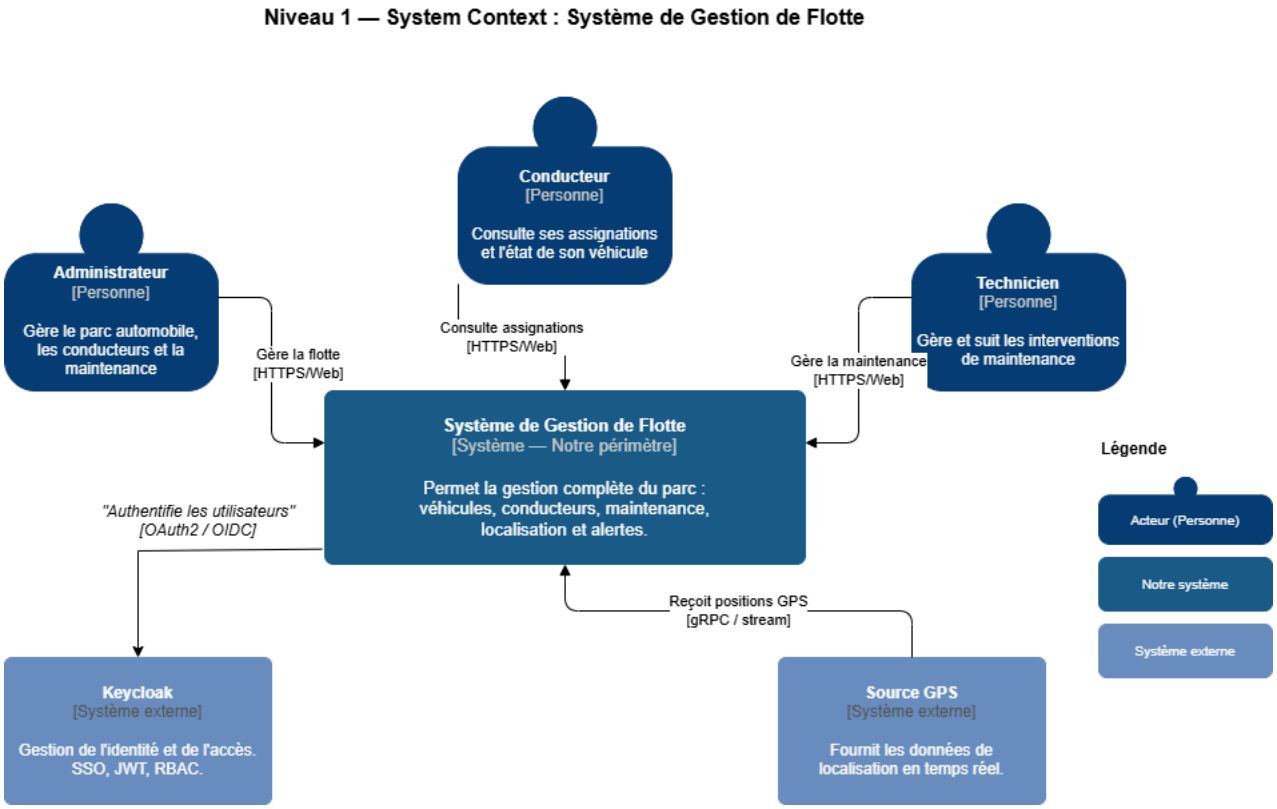
L'objectif de cette première semaine est de poser les fondations architecturales et techniques du projet avant tout développement. Chaque décision prise ici engage l'ensemble du projet.

2. Architecture Logicielle — Modèle C4

Nous avons adopté le modèle C4 (Simon Brown) pour documenter l'architecture de manière hiérarchique et progressive, du plus général au plus détaillé.

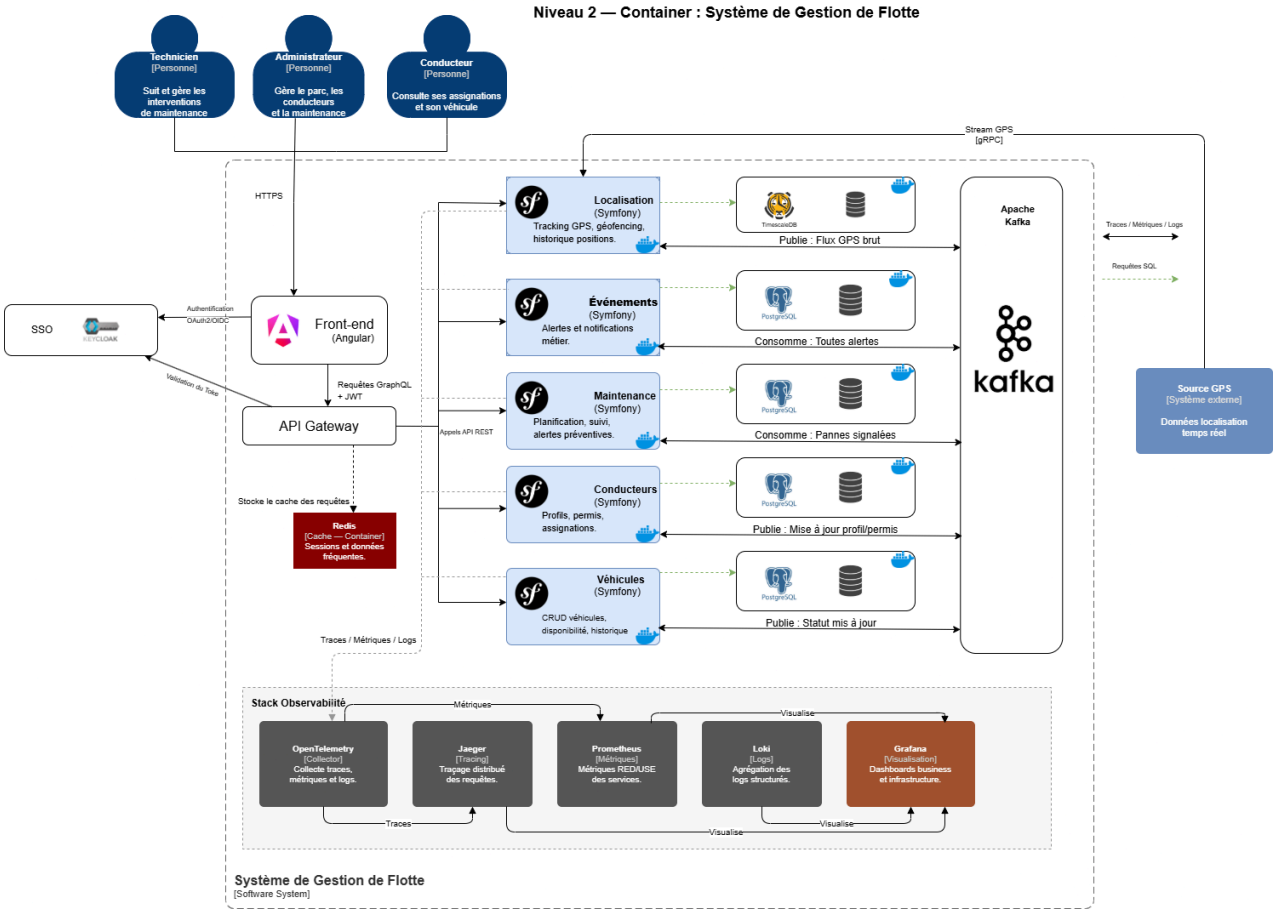
2.1 Contexte du Système — Niveau 1

Le système interagit avec trois types d'acteurs : Administrateur, Conducteur et Technicien. Il délègue l'authentification à Keycloak (SSO/OAuth2) et reçoit des données de localisation en temps réel via une source GPS externe.



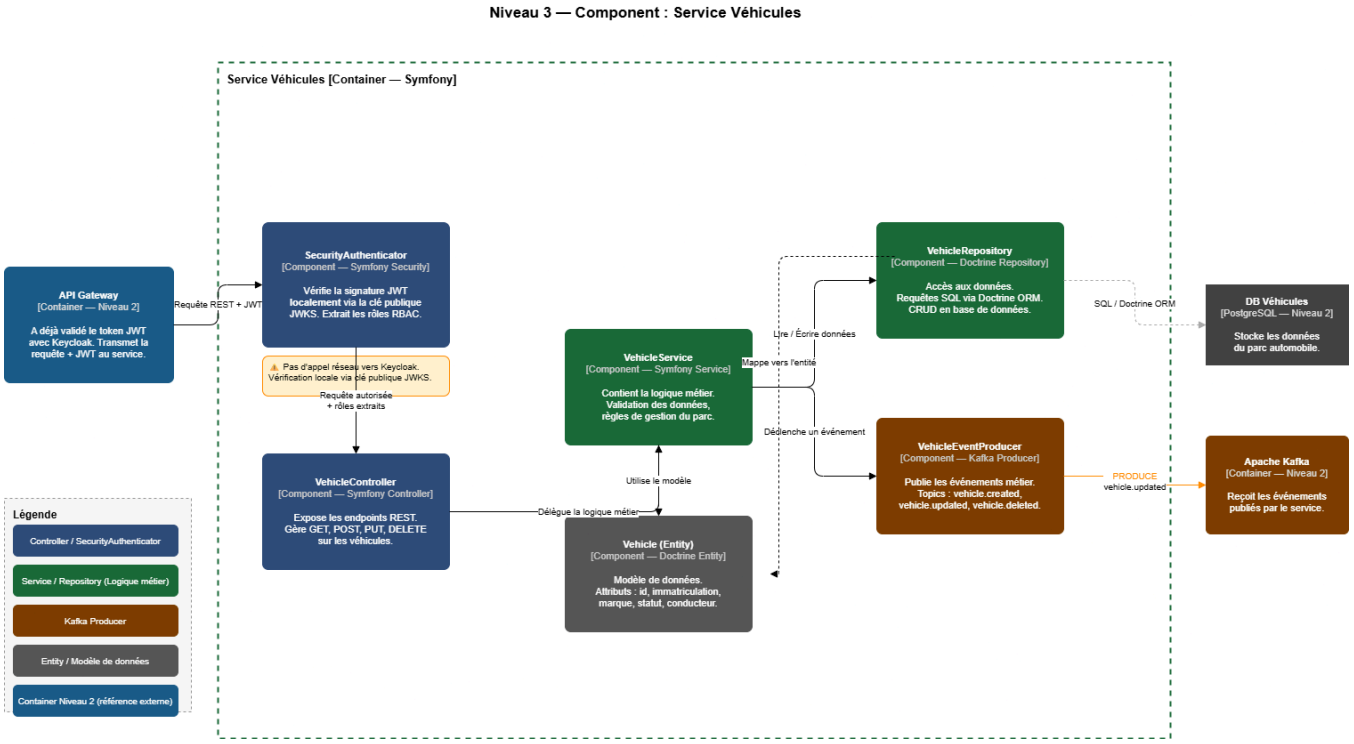
2.2 Conteneurs — Niveau 2

L'application est découpée en 5 microservices Symfony indépendants, chacun avec sa propre base de données (Database-per-Service pattern). Une API Gateway centralise les requêtes GraphQL, Redis assure le cache des sessions, et Apache Kafka gère la communication asynchrone inter-services.



2.3 Composants — Niveau 3 : Service Véhicules

Ce zoom illustre l'implémentation interne du Service Véhicules en Symfony. La sécurité est assurée par un Security Authentication qui vérifie la signature JWT localement via une clé publique JWKS — sans appel réseau vers Keycloak (déjà effectué par l'API Gateway).



3. Architecture Decision Records (ADR)

Les ADR tracent les décisions architecturales importantes et leur justification. Ils constituent une documentation vivante du projet.

ADR	Sujet	Décision	Justification	Impact
001	Messagerie	Apache Kafka	Persistance, rejeu et découplage des services	Tous les services
002	API	GraphQL Gateway	Agrégation flexible des données multi-services côté frontend	Frontend ↔ Services
003	Données GPS	TimescaleDB	Extension PostgreSQL optimisée pour les séries temporelles	Service Localisation
004	Sécurité	Keycloak	Centralise la gestion des identités et des rôles (SSO/OIDC)	Tous les services
005	Cache	Redis	Cache de session et réduction de la charge sur les bases de données	Frontend, API Gateway

4. Schémas BDD : Modèles Logiques de Données

- Chaque microservice dispose de sa propre base de données isolée (pattern Database-per-Service). Les clés étrangères sont logiques : elles ne créent pas de contraintes physiques entre services, car chaque base est indépendante.

– 1. Service Véhicules

Table principale du parc automobile. Contient tous les véhicules et leur état actuel.

TABLE : vehicules [PostgreSQL]

Colonne	Type	Contrainte	NULL	Description
id	UUID	**PK**	**NON**	Identifiant unique
immatriculation	VARCHAR(20)	UNIQUE	**NON**	Ex: AB-123-CD
marque	VARCHAR(50)	—	**NON**	Ex: Renault, Peugeot
modele	VARCHAR(50)	—	**NON**	Ex: Clio, 308
annee	INTEGER	—	OUI	Année de mise en service
statut	ENUM	disponible|en_mission|maint.	**NON**	État courant du véhicule
conducteur_id	UUID	LOGICAL FK → conducteurs.id	OUI	Conducteur assigné (nullable)
created_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date de création
updated_at	TIMESTAMP	—	OUI	Dernière mise à jour

– 2. Service Conducteurs

– Gère les profils des conducteurs, leurs permis et leur statut d'activité.

TABLE : conducteurs [PostgreSQL]

Colonne	Type	Contrainte	NULL	Description
id	UUID	**PK**	**NON**	Identifiant unique
nom	VARCHAR(50)	—	**NON**	Nom de famille
prenom	VARCHAR(50)	—	**NON**	Prénom
email	VARCHAR(100)	UNIQUE	**NON**	Email professionnel
telephone	VARCHAR(20)	—	OUI	Numéro de contact
numero_permis	VARCHAR(30)	UNIQUE	**NON**	Numéro de permis
date_expiration_permis	DATE	—	**NON**	Validité du permis
statut	ENUM	actif|inactif|suspendu	**NON**	État du conducteur
created_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date de création
updated_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date mise à jour

– 3. Service Maintenance

– Planification et suivi des interventions techniques sur les véhicules.

TABLE : Techniciens [PostgreSQL]

Colonne	Type	Contrainte	NULL	Description
id	UUID	**PK**	**NON**	Identifiant unique
nom	VARCHAR(50)	—	**NON**	Nom de famille
prenom	VARCHAR(50)	—	**NON**	Prénom
email	VARCHAR(100)	UNIQUE	**NON**	Email professionnel
telephone	VARCHAR(20)	—	OUI	Numéro de contact
statut	ENUM	actif|inactif	**NON**	État du conducteur
created_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date de création
updated_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date mise à jour

TABLE : interventions [PostgreSQL]

Colonne	Type	Contrainte	NULL	Description
id	UUID	**PK**	**NON**	Identifiant unique

vehicule_id	UUID	LOGICAL FK → vehicules.id	**NON**	Véhicule concerné
technicien_id	UUID	FK → techniciens.id	OUI	Technicien assigné
type_intervention	VARCHAR(100)	—	**NON**	Ex: vidange, freins
date_planifiee	DATE	—	**NON**	Date prévue
date_realisation	DATE	—	OUI	Date effective (NULL si pas encore fait)
statut	ENUM	planifie|en_cours|termine|annule	**NON**	Avancement
description	TEXT	—	OUI	Détails de l'intervention
created_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date de création
updated_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date mise à jour

– 4. Service Localisation

– Stockage des positions GPS en série temporelle. TimescaleDB partitionne automatiquement les données par intervalles de temps (hypertable).

TABLE : positions Hypertable TimescaleDB [TimescaleDB]

Colonne	Type	Contrainte	NULL	Description
id	UUID	**PK**	**NON**	Identifiant unique
vehicule_id	UUID	LOGICAL FK → vehicules.id	**NON**	Véhicule tracé
latitude	DECIMAL(10,8)	—	**NON**	Ex: 48.85341000
longitude	DECIMAL(11,8)	—	**NON**	Ex: 2.34880000
Vitesse	DECIMAL(5,2)	—	**Non**	Ex : 85.50 (en km/h)
time	TIMESTAMPTZ	INDEX HYPERTABLE	**NON**	Colonne de partitionnement temporel

– 5. Service Événements

– Journal centralisé des alertes et notifications générées par tous les services via Kafka.

TABLE : logs_events [PostgreSQL]

Colonne	Type	Contrainte	NULL	Description
id	UUID	**PK**	**NON**	Identifiant unique
type_event	VARCHAR(50)	—	**NON**	Ex: geofence_alert, maintenance_due
vehicule_id	UUID	LOGICAL FK → vehicules.id	OUI	Véhicule concerné (peut être NULL)

source_service	VARCHAR(50)	—	**NON**	Service émetteur (véhicules, localisation…)
message	TEXT	—	**NON**	Contenu de l'événement
niveau	ENUM	info|warning|critique	**NON**	Sévérité de l'alerte
date_eve	TIMESTAMP	—	**NON**	Date de l'événement
created_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date d'insertion en base
updated_at	TIMESTAMP	DEFAULT NOW()	**NON**	Date mise à jour

5. Topics Kafka

Topic	Producer	Consumer(s)	Description	Formats de messages
driver.assigned	Conducteurs	Véhicules, Événements	Assignation d'un conducteur à un véhicule	{ "event": "DRIVER_ASSIGNED", "timestamp": "2026-03-12T10:00:00Z", "payload": { "vehicle_id": "uuid-v123", "driver_id": "uuid-d456", "assigned_by": "uuid-admin-789" } }
vehicle.updated	Véhicules	Événements	Statut ou données du véhicule mis à jour	{ "event": "VEHICLE_STATUS_CHANGED", "payload": { "vehicle_id": "uuid-v123", "old_status": "disponible", "new_status": "en_mission" } }
maintenance.alert	Maintenance	Événements	Alerte d'intervention ou de panne signalée	{ "event": "MAINTENANCE_REQUIRED", "payload": { "vehicle_id": "uuid-v123", "priority": "high", "reason": "oil_change_overdue" } }
geofence.alert	Localisation	Événements	Sortie d'une zone géographique autorisée	{ "event": "GEOFENCE_VIOLATION", "payload": { "vehicle_id": "uuid-v123", "current_lat": 49.4431, "current_long": 1.0993, "zone": "Zone_A" } }

6. Contrats d'API

- **1. Service Véhicules**

Route	Méthode	Description

/vehicles	GET	Liste tous les véhicules avec filtres (marque, statut)
/vehicles	POST	Enregistre un nouveau véhicule dans la flotte.
/vehicles/{id}	GET	Récupère les détails complets d'un véhicule spécifique.
/vehicles/{id}	PUT	Met à jour les informations ou le statut d'un véhicule.
/vehicles/{id}	DELETE	Supprime un véhicule de l'inventaire.

- 2. Service Conducteurs

Route	Méthode	Description
/drivers	GET	Liste tous les conducteurs enregistrés
/drivers	POST	Crée un nouveau profil conducteur.
/drivers/{id}	GET	Récupère la fiche détaillée d'un conducteur.
/drivers/{id}	PUT	Modifie les coordonnées ou les informations du permis.
/drivers/{id}/status	PATCH	Change l'état d'activité (actif, suspendu, inactif).

- 3. Service Maintenance

Route	Méthode	Description
/interventions	GET	Liste toutes les interventions (planifiées, en cours, finies).
/interventions	POST	Crée une nouvelle demande de maintenance pour un véhicule.
/interventions/{id}	GET	Détails d'une intervention et rapport technique associé.
/interventions/{id}	PUT	Met à jour l'avancement ou assigne un technicien.
/technicians	GET	Liste les techniciens disponibles dans le service.

- 4. Service Localisation

Route	Méthode	Description
/locations/latest/{v_id}	GET	Récupère la dernière position connue d'un véhicule.
/locations/history/{v_id}	GET	Récupère l'historique des trajets sur une période donnée.
/geofences	GET	Liste les zones géographiques autorisées/interdites.
/geofences	POST	Définit une nouvelle zone de surveillance (geofencing).

- 5. Service Événements

Route	Méthode	Description
/events	GET	Liste tous les événements système et alertes métier.
/events/{id}	GET	Détails d'une alerte spécifique.
/events/vehicle/{v_id}	GET	Filtre tous les événements liés à un véhicule précis.
/events/{id}/ack	POST	Marque une alerte comme "lue" ou "traitée" par un manager.

7. Keycloak

- 1. Les Rôles

Rôle	Permissions	Services accessibles
Admin	Accès complet en lecture et écriture	Tous les services
Manager	Gestion de la flotte et des rapports	Véhicules, Conducteurs, Événements
Technicien	Gestion et suivi des interventions	Maintenance
Conducteur	Consultation de ses propres assignations	Véhicules (lecture), Localisation (lecture)

- 2. Le Realm

Le **Realm** est le conteneur principal de la configuration de sécurité.

- **Nom**: FleetManagement.
- **Description** : Système d'authentification centralisé pour l'entreprise (SSO, OAuth2, OIDC).
- **Rôle** : Il isole les utilisateurs et les accès de cette entreprise de tout autre système

- 3. Les Clients

Les **Clients** sont les applications autorisées à utiliser Keycloak pour l'authentification:

- **fleet-frontend** :
  - **Type** : Public
  - **Technologie** : Angular avec le Keycloak JS Adapter.
  - **Usage** : Permet aux employés (Conducteurs, Managers) de se connecter via l'interface web.
- **api-gateway** :
  - **Type** : Confidentiel.
  - **Technologie** : GraphQL Gateway.
  - **Usage** : Vérifie la validité des jetons JWT avant de laisser passer les requêtes vers les microservices Symfony.

8. Stack Technologique

Composant	Technologies
Backend	Symfony (PHP) — 5 microservices indépendants
Frontend	Angular — Microfrontend — Keycloak JS Adapter
API	REST + GraphQL (agrégation) + gRPC (streaming GPS)
Messagerie	Apache Kafka — communication asynchrone inter-services
Bases données	PostgreSQL (x4) + TimescaleDB (Localisation) + Redis (cache)
Sécurité	Keycloak — SSO, OAuth2/OIDC, JWT, RBAC
Observabilité	OpenTelemetry · Jaeger · Prometheus · Grafana · Loki
Déploiement	Docker · Kubernetes (Minikube) · Helm · GitHub Actions (CI/CD)

Document réalisé dans le cadre du Projet M1 GIL — Université de Rouen — 2025/2026